Minería de Datos

Clasificación

Algoritmos (Árboles, KNN, Naive Bayes, Reg. Logística)

Valentin Barriere / Mauricio Cerda / Cinthia Sánchez

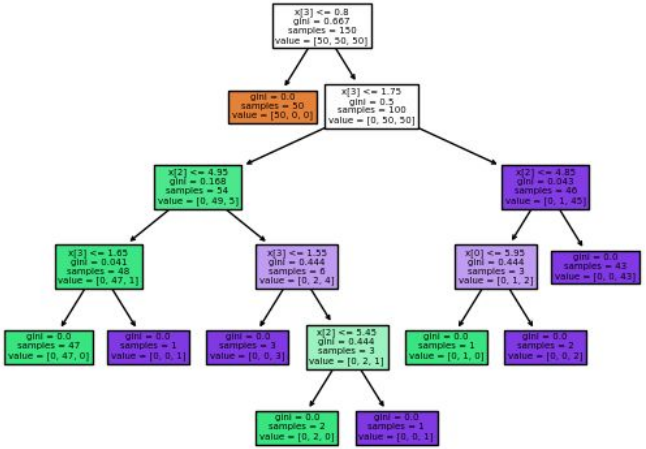
CC5205, Otoño 2026

Basado en las slides de Bárbara Poblete

Algoritmos de aprendizaje de máquinas

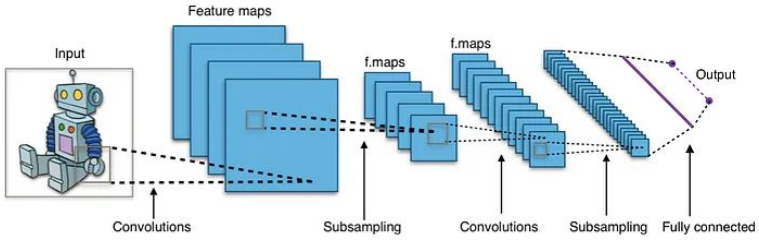
Algoritmos que pueden aprender de forma automática a través de datos. Esperamos que la máquina desarrolle su propia representación.

- Algoritmos tradicionales. Ej:



Árboles de decisión

- Algoritmos basados en redes neuronales. Ej:



Red Neuronal Convolucional (CNN)

Árbol de Decisión

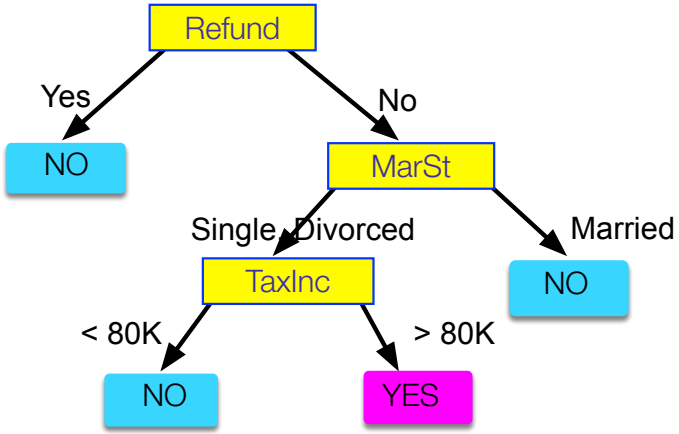
El árbol tiene tres tipos de nodos:

- **Nodo raíz:** no tiene arcos entrantes y tiene arcos salientes.
- **Nodos internos**: tienen exactamente un arco entrante y dos o más arcos salientes.
- **Nodos hoja o terminales**: tienen exactamente un arco entrante.

A cada nodo hoja se le asigna una etiqueta de clase.

Los nodos no terminales contienen tests sobre los atributos para separar los ejemplos.

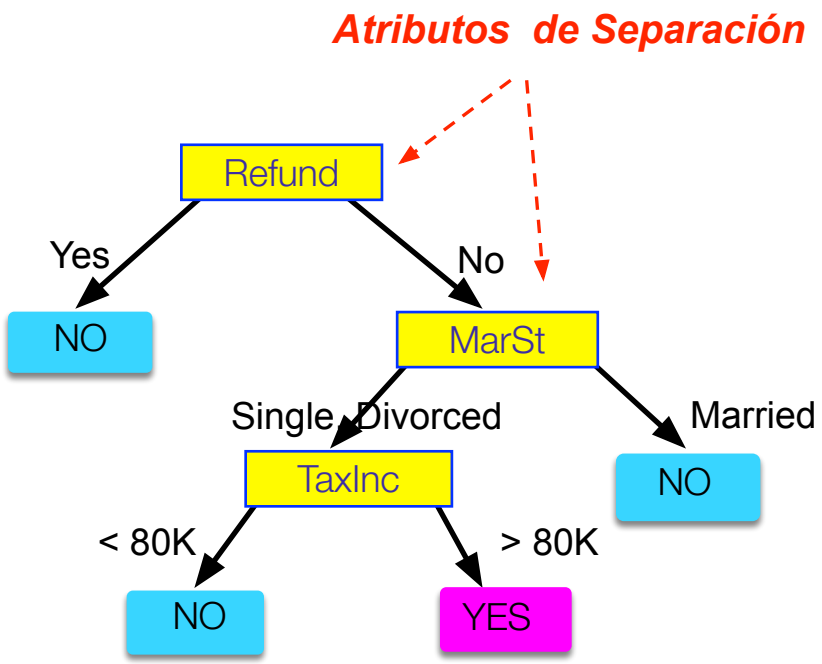
El árbol de decisión fragmenta el dataset de manera recursiva hasta asignar los ejemplos a una clase.



Árbol de Decisión

Tid	Refund	Marital Status	Taxable Income	Cheat
	categórica	categórica	numérica	Clase de interés
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Datos de Entrenamiento

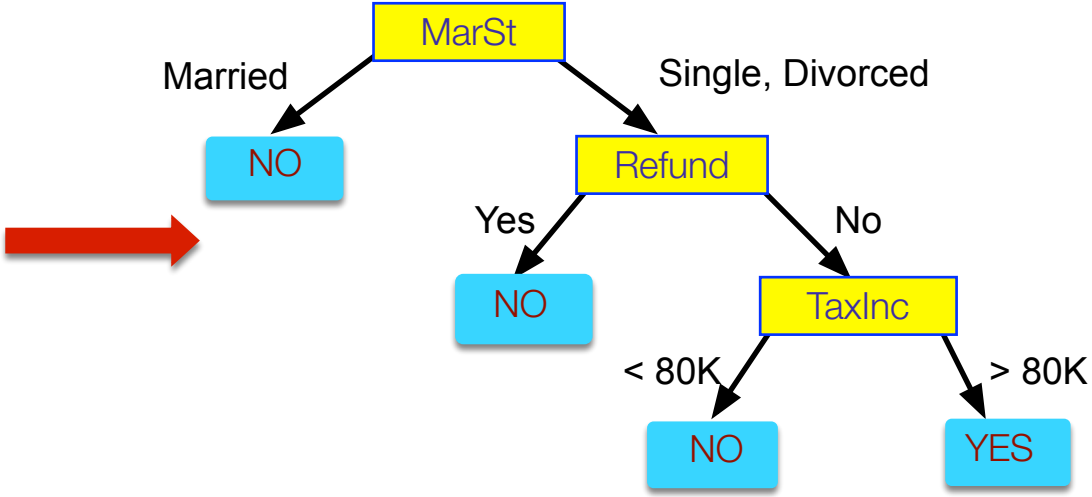


Modelo: Árbol de Decisión

Árbol de Decisión

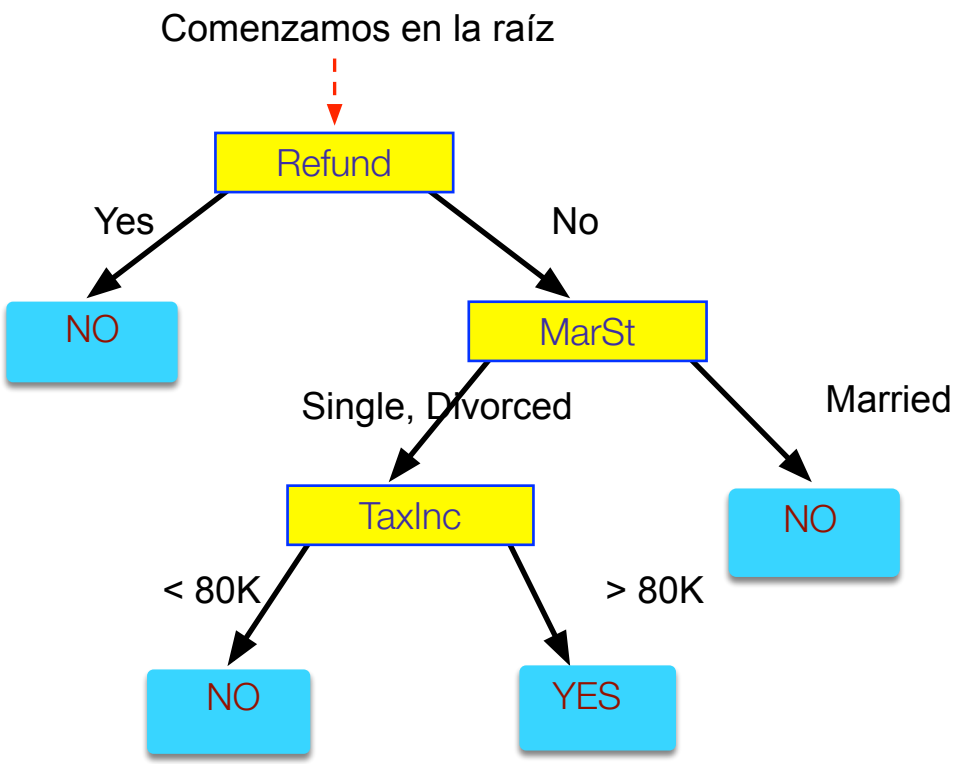
	categórica	categórica	numérica	Clase de interés
Tid	**Refund**	**Marital Status**	**Taxable Income**	**Cheat**
1	Yes	Single	125K	**No**
2	No	Married	100K	**No**
3	No	Single	70K	**No**
4	Yes	Married	120K	**No**
5	No	Divorced	95K	**Yes**
6	No	Married	60K	**No**
7	Yes	Divorced	220K	**No**
8	No	Single	85K	**Yes**
9	No	Married	75K	**No**
10	No	Single	90K	**Yes**

Datos de Entrenamiento



¡Puede existir más de un árbol que se ajuste a los datos!

Aplicamos el modelo



Dato de Evaluación

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Fáciles de interpretar
y visualizar

Construyendo un Árbol de Decisión

Estrategia: Top down (greedy) - Divide y vencerás recursiva

1. Seleccionar un atributo para el nodo raíz y crear rama para cada valor posible del atributo .
2. Dividir las instancias del dataset en subconjuntos, uno para cada rama que se extiende desde el nodo.
3. Repetir de forma recursiva para cada rama, utilizando sólo las instancias que llegan a ésta.
4. Detenerse cuando *todas* las instancias del nodo sean de la misma clase.

Criterio para escoger el mejor atributo

¿Qué atributo escojo?

- La idea es crear el árbol más pequeño posible.
- Heurística: escoge el atributo que produce nodos lo más “puros” posible.

El criterio más popular de pureza: **information gain**

- Information gain crece cuando crece la pureza promedio de los subconjuntos.

Estrategia: escoger el atributo que maximiza el valor de information gain.

Un árbol de decisión hace cortes perpendiculares a los ejes

Encontrar cortes que crean regiones *puras* (con ejemplos de una misma clase).
-> Necesitamos medir la pureza

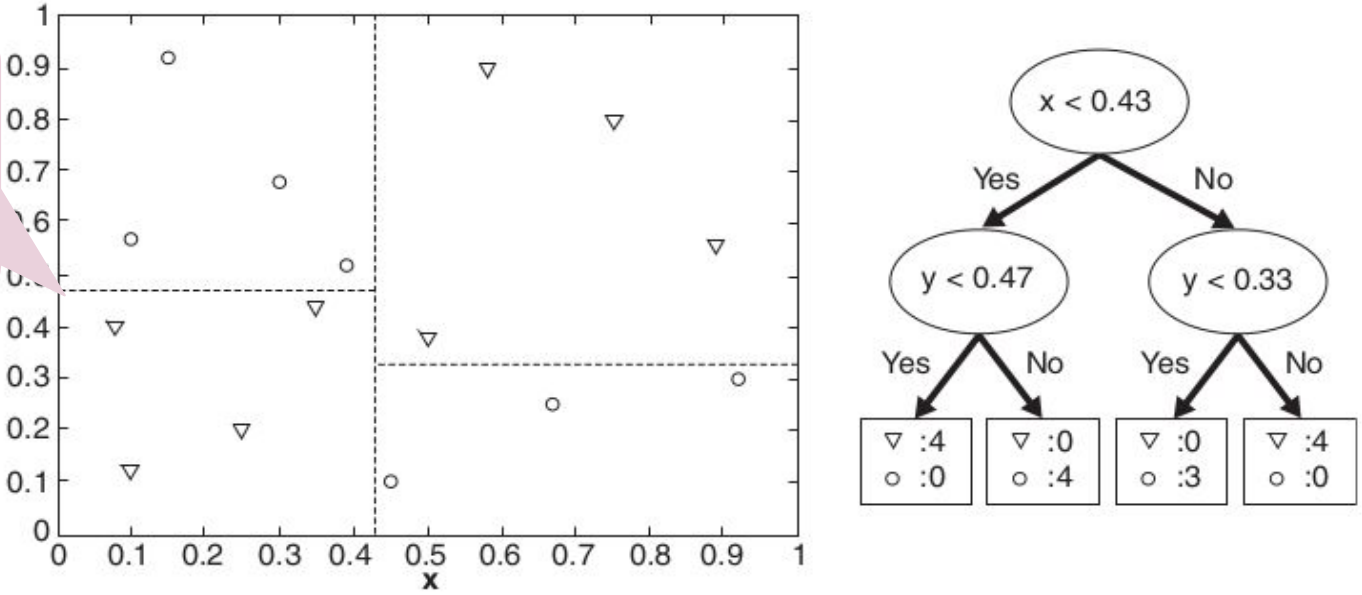


Figure 3.20. Example of a decision tree and its decision boundaries for a two-dimensional data set.

Criterio para escoger el mejor split: Función objetivo

Node training

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta} \in \mathcal{T}} I$$

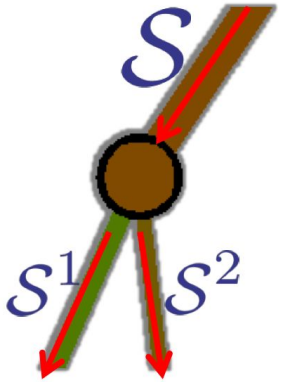
Information gain

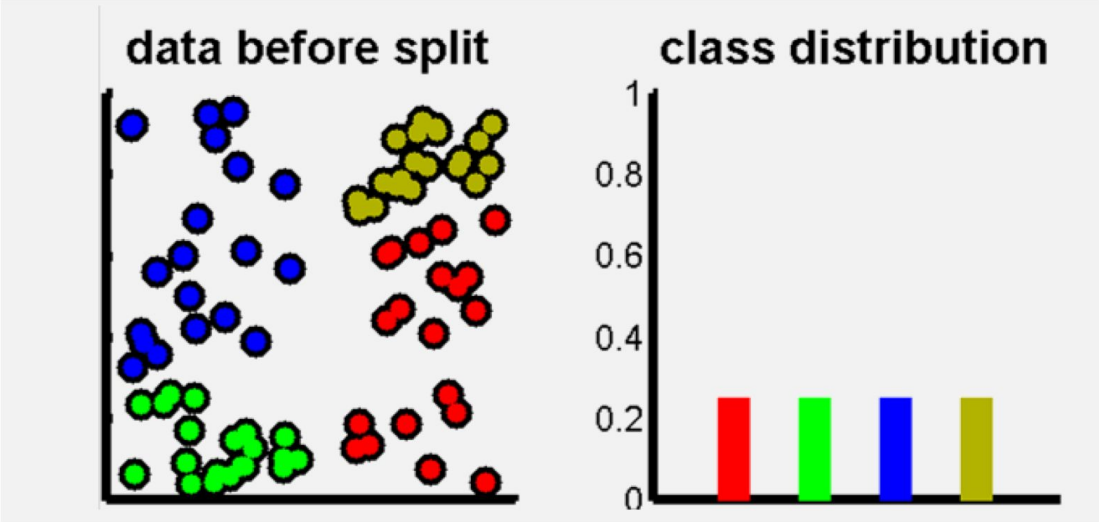
$$I = H(\mathcal{S}) - \sum_{i \in \{1,2\}} \frac{|\mathcal{S}^i|}{|\mathcal{S}|} H(\mathcal{S}^i)$$

Shannon's entropy

$$H(\mathcal{S}) = -\sum_{c \in \mathcal{C}} p(c) \log(p(c))$$

Information gain: Info antes del split – info después del split





Shannon entropy

$$H(\mathcal{S}) = -\sum_{c \in \mathcal{C}} p(c) \log(p(c))$$

- Mide la incertidumbre promedio de una variable aleatoria
- Número de bits para comunicar un mensaje H(S)
- Si tuviéramos 5 símbolos {A,B,C,D,E} con probabilidades de aparición {0.5, 0.2, 0.1, 0.1, 0.1}

$$H(X) = -[(0.5\log_2 0.5 + 0.2\log_2 0.2 + (0.1\log_2 0.1)*3)]$$
$$H(X) = -[-0.5 + (-0.46438) + (-0.9965)]$$
$$H(X) = -[-1.9]$$
$$H(X) = 1.9 \text{ bits/símbolo}$$

- Si ahora transmito AAAAABBCDE (10 símbolos), requiero 20 bits para codificarlos.

Función objetivo

Node training

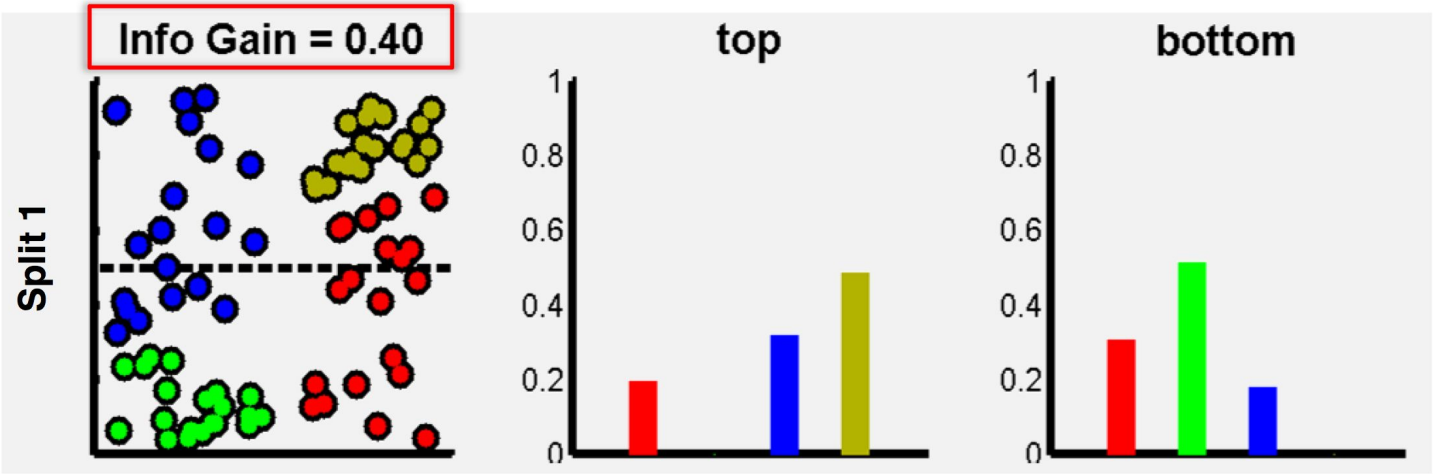
$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta} \in \mathcal{T}} I$$

Information gain

$$I = H(\mathcal{S}) - \sum_{i \in \{1,2\}} \frac{|\mathcal{S}^i|}{|\mathcal{S}|} H(\mathcal{S}^i)$$

Shannon's entropy

$$H(\mathcal{S}) = -\sum_{c \in \mathcal{C}} p(c) \log(p(c))$$



Función objetivo

Node training

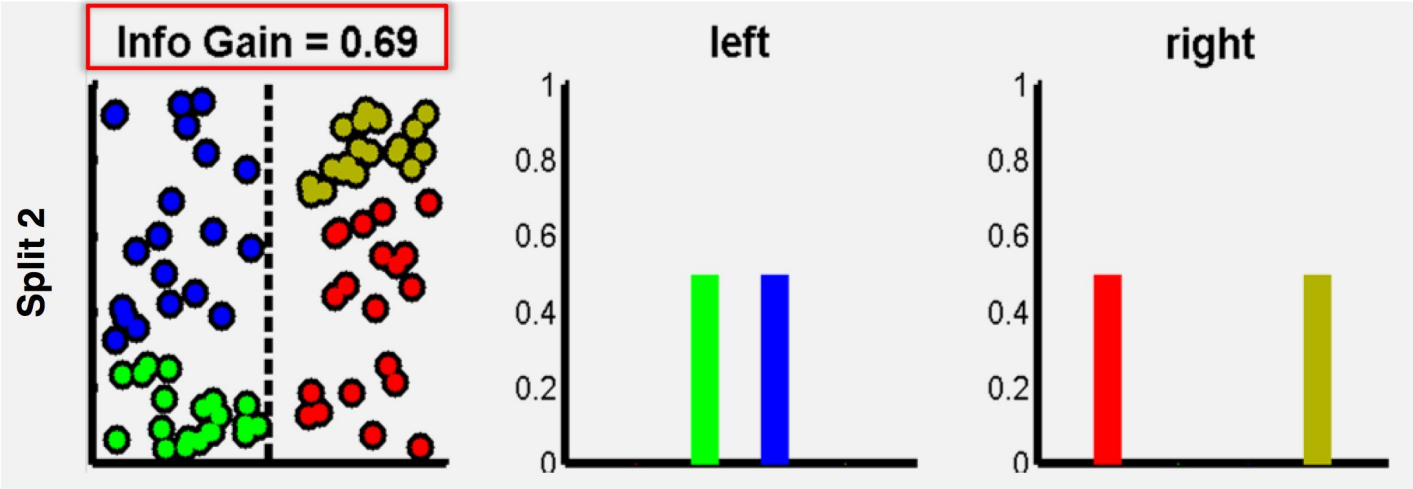
$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}\in\mathcal{T}} I$$

Information gain

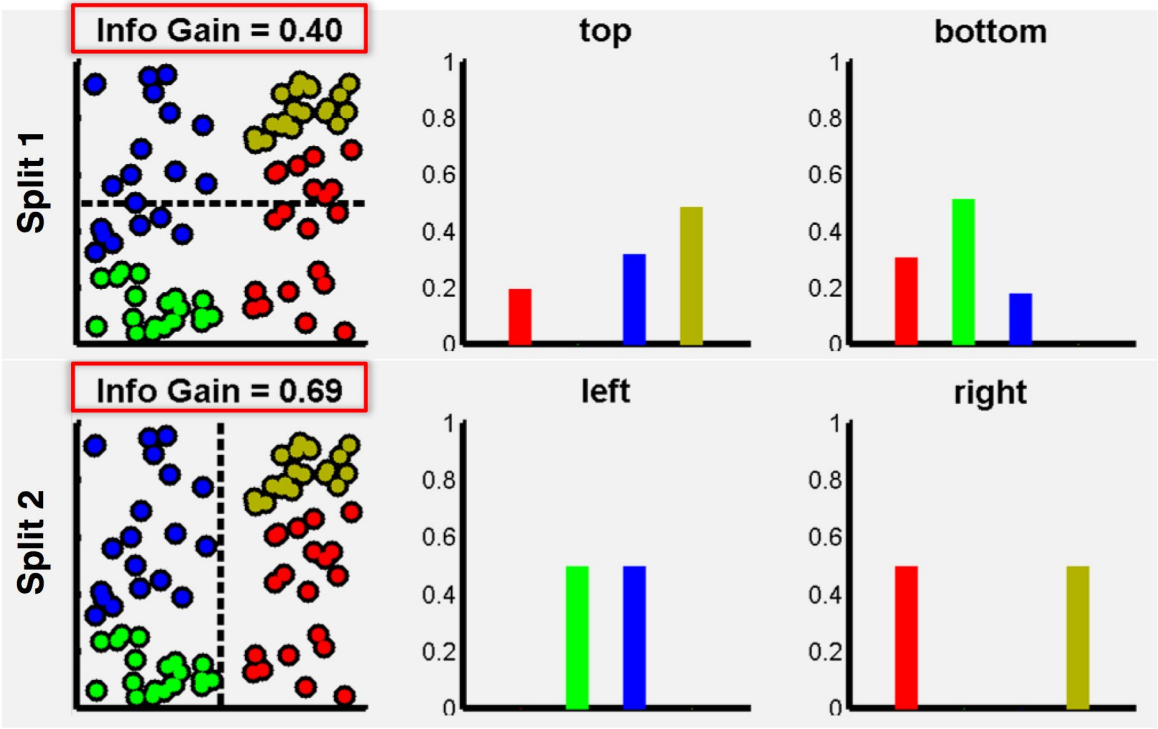
$$I = H(\mathcal{S}) - \sum_{i\in\{1,2\}} \frac{|\mathcal{S}^i|}{|\mathcal{S}|} H(\mathcal{S}^i)$$

Shannon's entropy

$$H(\mathcal{S}) = -\sum_{c\in\mathcal{C}} p(c)\log(p(c))$$

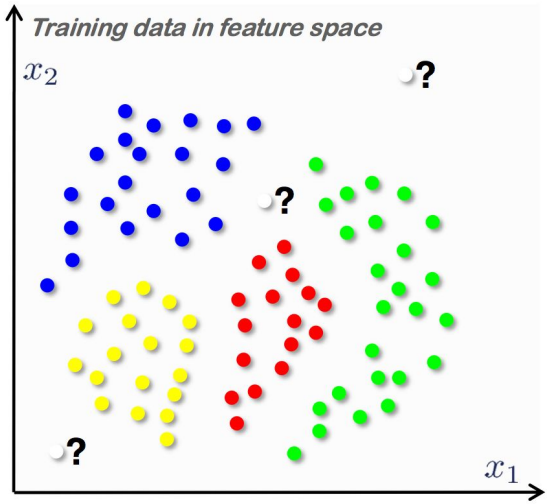


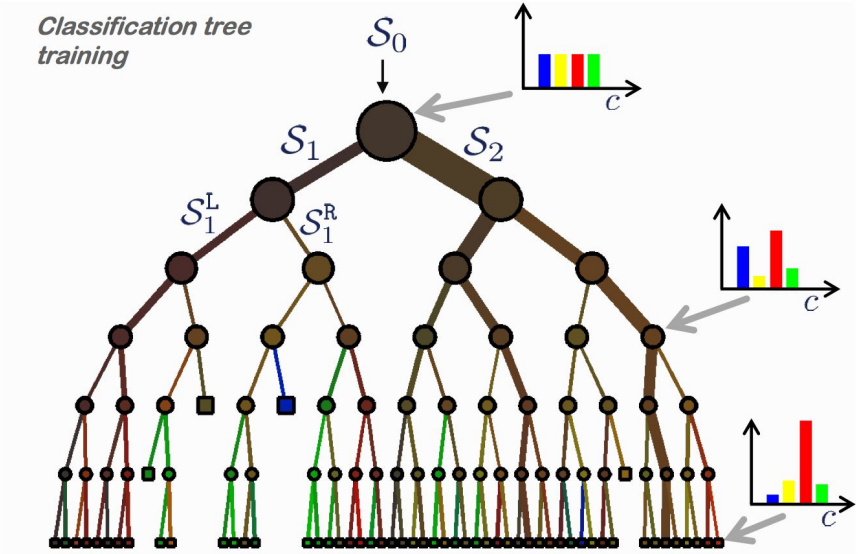
Función objetivo



Función objetivo

<u>Objetivo</u>: reducir la incertidumbre





Árbol de Decisión Resultante

Nota: no todas las hojas tienen que ser puras; a veces instancias idénticas tienen clases diferentes.

→ El splitting termina cuando los datos no se pueden seguir particionando.

Se puede exigir un mínimo número de instancias en la hoja para evitar sobreajuste.

Criterios de parada:

- Profundidad máxima.
- Función objetivo menor a un cierto umbral.
- Número de ejemplos menor a un cierto número.

Comentarios

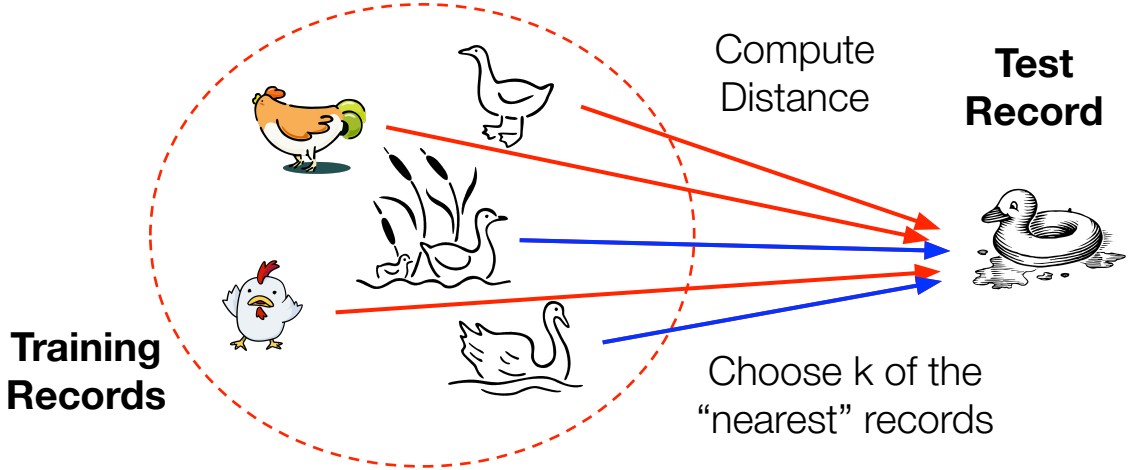
- Los atributos numéricos son discretizados, escogiendo la partición que maximice information gain.
- Existen otras métricas para medir pureza distintas a entropía como el índice de **Gini** =1-Pr (Sacar dos ejemplos de la misma clase).
- Para evitar sobre-ajuste los árboles pueden ser podados (se eliminan ramas que alcanzan muy pocos ejemplos).
- La gran ventaja de los árboles es la **interpretabilidad**.

Clasificador KNN

Nearest Neighbor Classifier (o k-nn): Clasificador basado en **instancias**. Conocido como **lazy**.

- Usa los k puntos más cercanos (nearest neighbors) para realizar la clasificación

Idea: If it walks like a duck, quacks like a duck, then it's probably a duck.



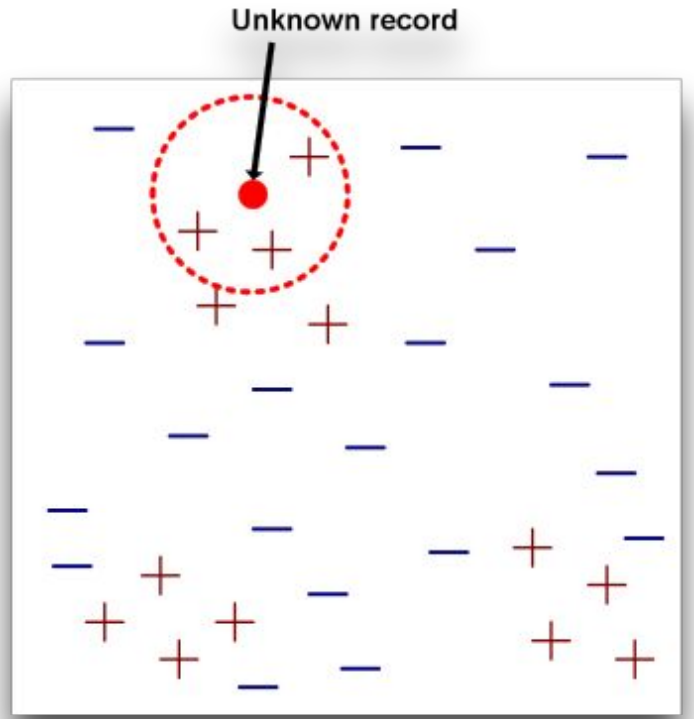
Clasificador KNN

Necesita 3 cosas:

- Set de **records/ejemplos** almacenados.
- **Métrica de distancia** para calcular entre records.
- Valor **k**, el número de vecinos cercanos a obtener.

Para clasificar un ejemplo nuevo:

1. Calcular la distancia a los ejemplos almacenados.
2. Identificar k nearest neighbors .
3. Utilizar la clase de los knn para asignar la clase al record nuevo (e.j. voto de la mayoría).



Métricas de distancia

Para atributos númericos usamos la distancia **euclidiana**:

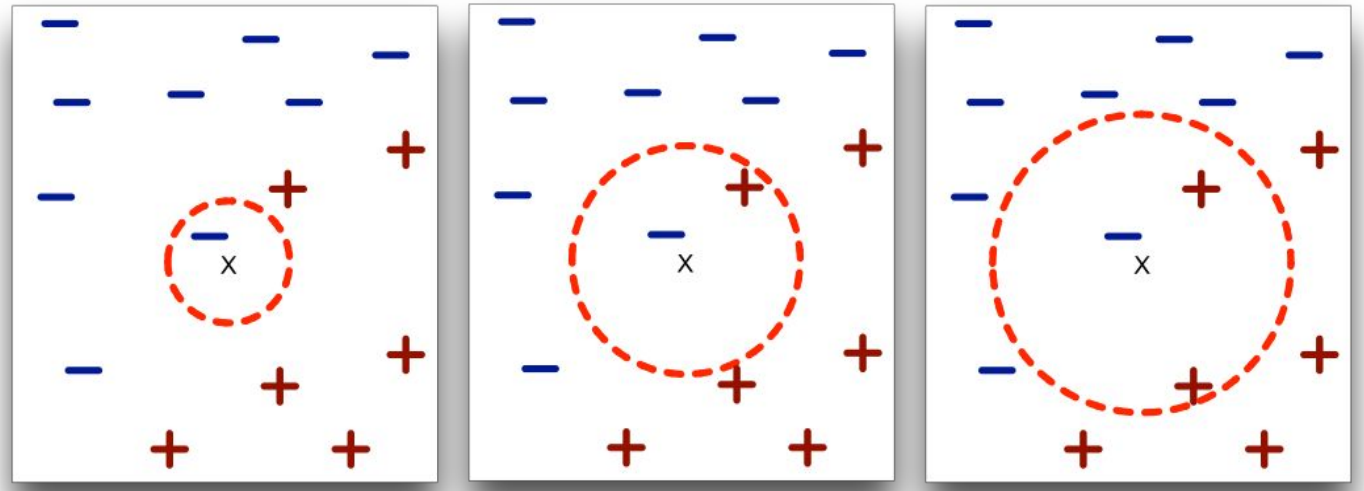
$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2},$$

Una versión más general es la distancia de **Minkowsky** (r=1 => distancia Manhattan, r=2 => distancia euclidiana)

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}$$

-> *Es muy importante que los atributos estén normalizados.*

Definición de NN

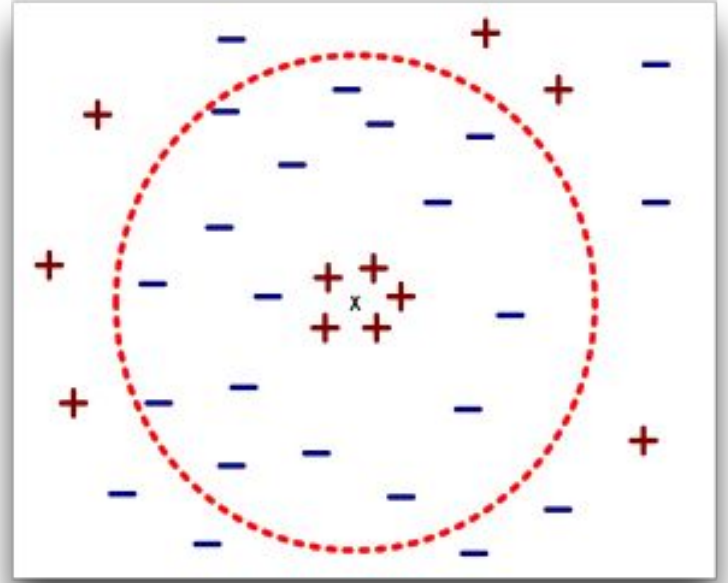


(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

K-NN de un record x son los puntos que tienen las k menores distancias a x

Eligiendo el valor de K

- k muy pequeño es susceptible a ruido
- k muy grande puede incluir puntos de otra clase



Clasificación kNN

Los clasificadores k-NN son **lazy learners**.

- No construyen modelos explícitos, es más flexible ya que no necesita comprometerse con un modelo global a priori.
    - Al contrario de los **eager learners** como los árboles de decisión o clasificadores basados en reglas.
- Es independiente del nro. de clases.
- La clasificación es más costosa (memoria y tiempo).

Comentarios

- Cuando los datos tienen una alta dimensionalidad, las técnicas basadas en distancias (KNN, K-means) están sujeta a la Maldición de la Dimensionalidad.
- Fenómeno en que muchos tipos de análisis de datos se vuelven significativamente más difíciles a medida que aumenta la dimensionalidad de los datos.
- Para la clasificación, esto puede significar que no haya suficientes ejemplos para crear un modelo que asigne de forma confiable una clase a todos los ejemplos posibles.

Clasificación Basada en Naïve Bayes

- Modelo que busca **modelar la relación probabilística** entre atributos y clase.
- Familia de modelos basados en Bayes.
- Modelo generativo, asume una distribución conjunta entre X e Y.
- Supuesto: atributos independientes dado la clase (**naive assumption**).

Clasificador Bayesiano

Esquema probabilístico para resolver problemas de clasificación.

- Probabilidad condicional:

$$P(C \mid A) = \frac{P(A,C)}{P(A)}$$

$$P(A \mid C) = \frac{P(A,C)}{P(C)}$$

- Teorema de Bayes:

$$P(C \mid A) = \frac{P(A \mid C)P(C)}{P(A)}$$

Ejemplo Teorema de Bayes

Dado:

- Un doctor sabe que la meningitis produce rigidez de cuello el 50% de las veces.
- La probabilidad previa de que cualquier paciente tenga meningitis es 1/50,000.
- La probabilidad previa de que cualquier paciente tenga rigidez en el cuello es de 1/20.

¿Si un paciente tiene el cuello rígido, cuál es la probabilidad de que tenga meningitis?

$$P(M \mid S) = \frac{P(S \mid M)P(M)}{P(S)} = \frac{0.5 \times 1/50000}{1/20} = 0.0002$$

Clasificador Naïve Bayes

- Considerar cada atributo como variable condicionalmente independiente de la clase (eso es “naive”).
- Dado un record con atributos (A_1, A_2,…, A_n).
  - La meta es predecir la clase C.
  - Específicamente queremos encontrar el C que maximice $P(C| A_1, A_2,\ldots,A_n)$.
- ¿Podemos estimar $P(C \mid A_1, A_2,\ldots,A_n)$ directamente de los datos?

Clasificador Naïve Bayes

Aproximación:

- Computar la probabilidad posterior $P(C \mid A_1, A_2, \ldots, A_n)$ para todos los valores de C usando el Teorema de Bayes.

$$\boxed{P(C \mid A_1 A_2 \ldots A_n) = \frac{P(A_1 A_2 \ldots A_n \mid C) P(C)}{P(A_1 A_2 \ldots A_n)}}$$

- Elegir un valor de C que maximice $P(C \mid A_1, A_2, \ldots, A_n)$.
- Equivalente a elegir un valor de C que maximice $P(A_1, A_2, \ldots, A_n|C)\ P(C)$.
- Esto es porque el numerador $P(A_1, A_2, \ldots, A_n)$ es constante para todas las clases.

Clasificador Naïve Bayes

- Asume independencia entre los atributos A_i cuando la clase está dada (independencia condicional):
- $P(A_1, A_2, \ldots, A_n \mid C) = P(A_1 \mid C_j)\, P(A_2 \mid C_j) \ldots P(A_n \mid C_j)$.
- Se puede estimar $P(A_i \mid C_j)$ para todos los A_i y C_j.
- Un punto nuevo A, se clasifica como C_j si $P(C_j)\ \Pi\ P(A_i \mid C_j)$ es máxima (en comparación con otros valores de C).

¿Cómo estimar probabilidades a partir de los datos?

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

¿Cómo calculamos P(Aik=b|Ck) cuando el atributo Ai es numérico?
-> Una opción es discretizar el atributo y proceder de la forma anterior.

- Clase: $P(C_k) = \frac{count(C_k)}{N}$
  - e.g., P(No) = 7/10,
    P(Yes) = 3/10
- Para atributos discretos:

$$P(A_i = b|C_k) = \frac{count(A_{ik} = b)}{count(C_k)}$$

  - donde count(A_{ik}=b) es el número de instancias que tiene el valor b para el atributo A_i y que pertenecen a la clase C_k
  - Ejemplos:
  - P(Status =Married|No) = 4/7
    P(Refund= Yes |Yes) = 0

Laplace Smoothing

$P(C \mid A_1, A_2, \ldots, A_n)$ se puede ir a cero cuando $|A_{ik}=b| = 0$ (cuando para alguna clase Ck no hay ningún ejemplo con $A_i=b$).

Le asignaría probabilidad cero a la clase Ck a cualquier ejemplo con $|A_{ik}=b| = 0$, ignorando el valor de los otros atributos (las probabilidades se multiplican).

Laplace Smoothing: soluciona el problema sumándole 1 a todos los conteos para que ninguna probabilidad quede en cero:

$$P(A_i = b|C_k) = \frac{count(A_{ik} = b) + 1}{count(C_K) + values(A_i)}$$

Donde values(Ai) es la cantidad de categorías del atributo Ai.

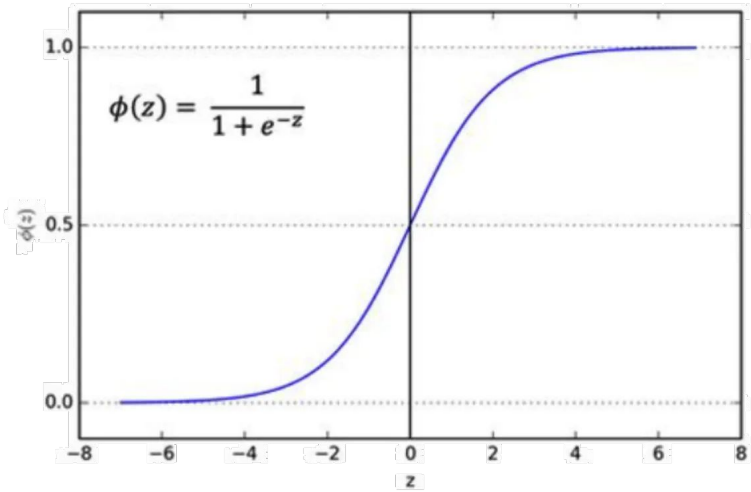
Con Laplace smoothing P(Status =Married|No) = (4+1)/(7+3)

Comentarios

- Es robusto ante puntos de ruido aislados.
- Maneja valores faltantes ignorando la instancia durante los cálculos de estimación de probabilidades.
- Robusto a atributos irrelevantes (afectan de igual manera a todas las clases).
- El supuesto de independencia entre atributos puede no ser cierto en todos los casos.
- Las redes Bayesianas o los modelos gráficos dirigidos permiten hacer modelos probabilísticos con supuestos de independencia menos restrictivos.

Regresión Logística

- Modelo lineal que examina la relación entre las variables predictoras (independientes) y una variable objetivo (dependiente).
- A pesar de su nombre, es un algoritmo de clasificación.
- Convierte una salida lineal $\mathbf{w}^T\mathbf{x} + b$ (o “distancia” en el espacio de características) en una probabilidad entre 0 y 1, mediante la función sigmoide (o softmax para multiclase).
- El umbral estándar de decisión es 0,5.

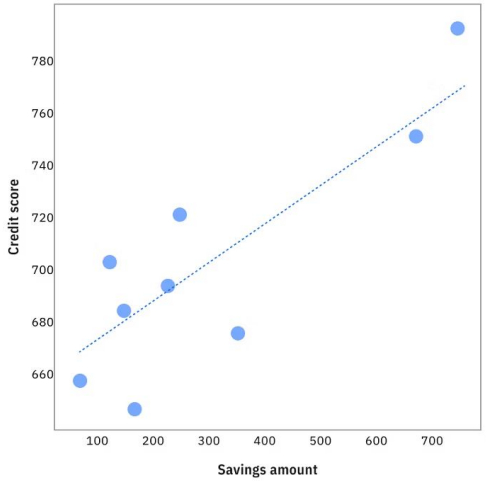


Regresión Logística

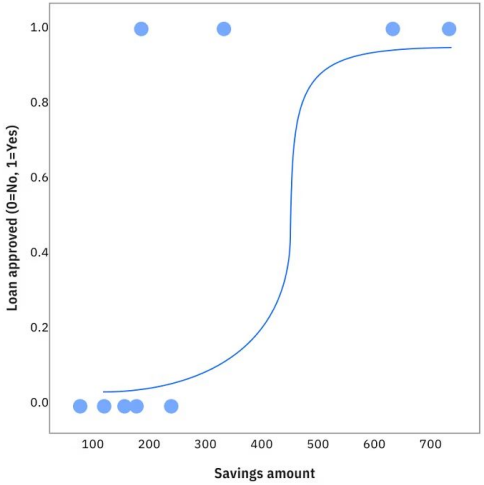
Consideremos una regresión lineal **[-Inf, +Inf]** donde queremos predecir la puntuación crediticia de alguien en función de sus ahorros.

Si queremos representar la **probabilidad [0,1]** de aprobar o no aprobar, aplicamos la logística

$$Y_{creditscore} = \beta_0 + \beta_1 X_{savings}$$



$$Y_{approval} = \beta_0 + \beta_1 X_{savings}$$



https://www.ibm.com/es-es/think/topics/logistic-regression

Regresión Logística

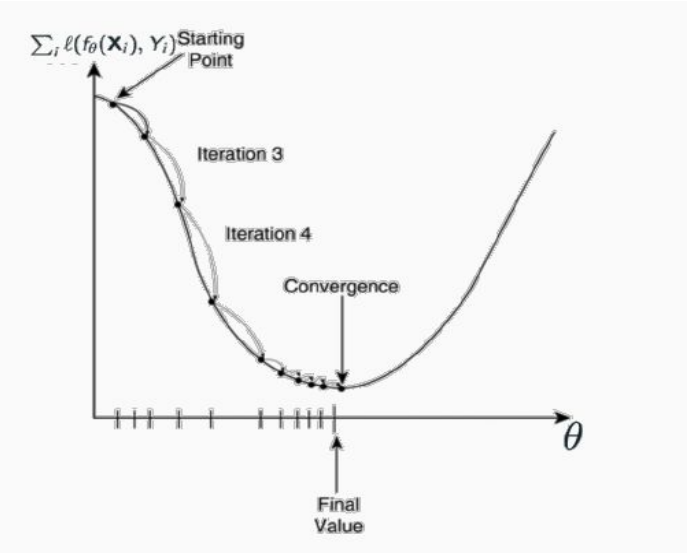
Función de Costo:

Para entrenar este modelo, necesitamos una función convexa fácil de optimizar. Una función muy utilizada es la *Entropía Cruzada (Cross-Entropy Loss)*.

Minimizando esta función sobre nuestro conjunto de entrenamiento, encontraremos los parámetros que mejor separan las clases.

Entrenamiento y Optimización:

Actualizamos iterativamente los parámetros del modelo calculando el gradiente de nuestra función de costo con *Descenso del Gradiente Estocástico (SGD).*



Tipos de Regresión Logística

Regresión Logística Binaria

- Dos resultados posibles. Ej: Spam o No Spam.
- Utiliza la función **Sigmoide** para calcular la probabilidad ***P(Y=1)***, separando las clases típicamente en un umbral de 0.5.

Regresión Logística Multinomial

- Para 3 o más clases (nominales). Ej: Predecir si un cliente elegirá Producto A, B o C.
- Utiliza la función **Softmax** para asignar una probabilidad a cada clase (que en total suman 1) y elige la más alta.

Comentarios

- Es rápida de entrenar, no requiere enormes cantidades de datos y es altamente interpretable (qué peso tiene cada variable en la decisión final).
- No sólo entrega una etiqueta ("Spam"), sino la probabilidad matemática ("98% de probabilidad de ser Spam"). Esto permite ajustar el umbral de decisión según el problema.
- El puente hacia el Deep Learning: La matemática (la función Sigmoide para decisiones binarias y Softmax para múltiples clases) es la misma que encontrarán en la capa final de las redes neuronales modernas.

Comparativa

Algoritmo	Principales Ventajas	Principales Desventajas	Casos de Uso Ideales
Árboles de Decisión	Alta interpretabilidad; no requiere escalado de atributos; maneja bien datos mixtos.	Muy propenso al sobreajuste; inestable ante pequeñas variaciones en los datos.	Reglas de negocio transparentes; explicabilidad estricta; datos estructurados tabulares.
KNN	Simple e intuitivo; no asume ninguna distribución subyacente ("lazy learner").	Inferencia lenta (calcula distancias contra todo el dataset); sufre la "maldición de la dimensionalidad".	Datasets pequeños; sistemas de recomendación básicos; reconocimiento de patrones espaciales.
Naïve Bayes	Entrenamiento rápido; escalable; requiere pocos datos para estimar probabilidades.	Asume independencia entre características (fuerte y poco realista)	Clasificación de texto (Spam, NLP); análisis de sentimientos; sistemas en tiempo real.
Regresión Logística	Muy rápida; coeficientes altamente interpretables; entrega probabilidades.	Asume separabilidad lineal estricta; sensible a valores atípicos.	Riesgo crediticio; diagnósticos médicos; *baselines* fuertes para clasificación binaria.

Conclusiones

- Hemos visto distintos algoritmos de clasificación, que siguen distintos paradigmas.
- Algunos de estos modelos se pueden usar tanto para regresión como para clasificación.
- Todos pueden funcionar bien dependiendo del problema. A priori, uno nunca sabe qué modelo funcionará mejor (*no free lunch theorem*)
- Es necesario experimentar para determinar qué modelo funciona mejor en cada caso.

Parte práctica

Veremos:

- Carga de datos
- Partición de datos en train y test
- Entrenamiento y evaluación del modelo (arbol de decisión)
  - Holdout
  - Cross-validation
- Visualización de matriz de confusión
- Seleccionar hiperparámetros
- Recomendaciones de lectura (baselines, codificación de atributos categóricos, balanceo de datos)

https://colab.research.google.com/drive/1JnsxfXMjyqEQff5b_jtE8w0E8fLVIM_M